

# Communication-avoiding solvers for the sea-surface-height system in FESOM2/Kokkos

Interim report

6 August 2026 — Levante (128-core CPU nodes, A100 GPU nodes)

## 1. Scope

The barotropic sea-surface-height (SSH) solve is the model’s synchronisation bottleneck. Preconditioned conjugate gradients perform two global reductions and two halo exchanges per iteration, and their cost grows as a mesh is spread over more cores. We implemented four alternatives behind a runtime switch — Chronopoulos–Gear CG (`cg2`), pipelined CG (`pipecg`), PIPECG-OATI (`oati`) and Chebyshev P-CSI (`pcsi`) — and measured them on four meshes and both backends. The certified baseline remains bit-identical when the switch is unset, which two byte-level gates confirm.

Every timing reported here comes from a same-allocation A/B against the baseline solver, minimum of two repetitions, 300 steps, with all four legs verified to have fired no fallback. A leg that falls back is a mixture of the variant and the baseline and is not an A/B point; this is the criterion under which an earlier set of fArc results was withdrawn.

## 2. Two defects had to be repaired first

### 2.1. The preconditioner is not symmetric

The preconditioner inherited from FESOM’s `solver.F90` is  $D^{-1}C$  with  $C$  symmetric. The leading  $1/d_i$  breaks symmetry, with a measured defect ratio of 0.62–0.74 across three meshes. Every CG–CG variant replaces the explicit inner product  $(p, Ap)$  with a recurrence that is valid only for a symmetric preconditioner, so the step length came out wrong by 21.8 % and most solves diverged. Symmetrising to  $D^{-1/2}CD^{-1/2}$  removes the error.

The same repair shortens the *existing* solver by about 7 % in iterations (126 to 117 on CORE2), so correcting `solver.F90` would benefit the Fortran model independently of anything else here.

### 2.2. The fallback guard was aborting healthy solves

Our own guard declares failure after 20 consecutive iterations without a 0.1 % residual drop. On fArc it fired at iteration 107 of solves that the baseline completes in 212, and it was the reason every fArc result was withdrawn. Re-running the identical dumped system with the guard widened, `cg2` converges at iteration 205 — six *fewer* than the baseline’s 211. The residual falls again three iterations after the point where the guard had given up.

The threshold is below what the reference method itself requires: baseline CG’s own longest plateau on fArc is 21 iterations. Residual non-monotonicity is ordinary CG behaviour, since only the  $A$ -norm of the error decreases monotonically, and on a system with condition number 843 a 20-iteration plateau is unremarkable. Baseline CG never reported a fallback for a simpler reason than robustness: it carries no guard at all, so its counter cannot increment.

## 3. Results

### 3.1. The payoff tracks the SSH share of the step

Figure 1(a) collects every valid configuration. The whole-step gain follows the share of the step the solve occupies, which varies from 0.8 % (DARS at 512 CPU ranks) to 37.7 % (fArc on 8 GPU nodes). Below roughly 5 % the effect is within measurement noise; above it the gain grows steadily.

That share is not a property of the mesh but of how thinly it is spread. On CORE2 it rises from 2.5 % at 128 ranks to 18.9 % at 864. The solve is also the only part of the model that anti-scales: from 128 to 864 ranks everything else speeds up  $5.40\times$  (79 % parallel efficiency) while the solve becomes 41 % *slower* in absolute terms. The solvers therefore pay most exactly where the model

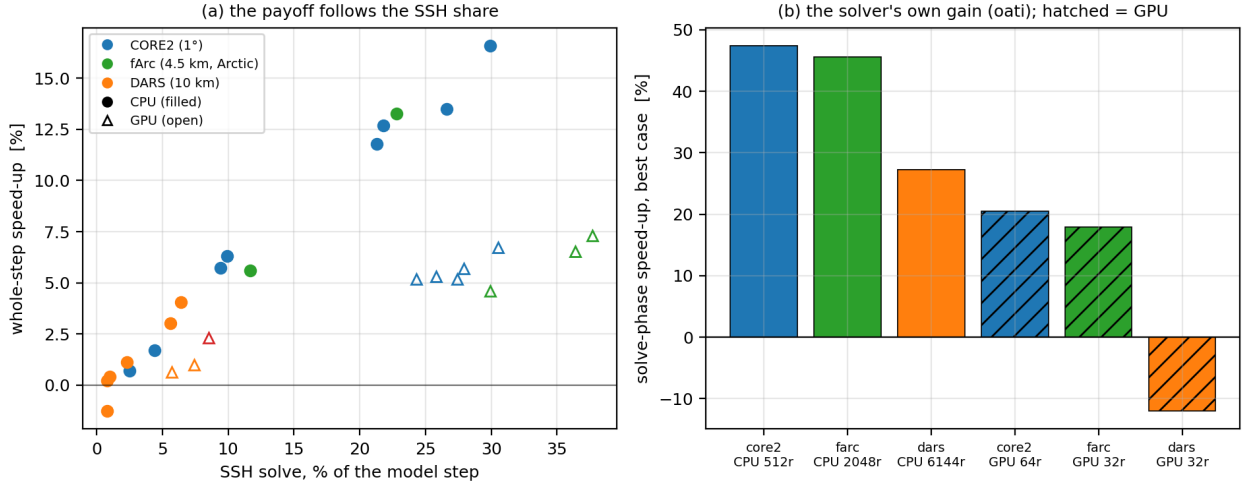


Figure 1: **(a)** Whole-step speed-up against the share of the step spent in the SSH solve, one point per configuration, best solver at that configuration. Filled circles are CPU, open triangles GPU. **(b)** The speed-up of the solve phase itself for `oati`; hatched bars are GPU. The two backends separate in (b), which is what makes them separate in (a): on CORE2 and fArc the solve is 46–47% faster on CPU but only 18–21% faster on GPU. On DARS, where the solve is a small share of the step, it is 27% faster on CPU and 12% slower on GPU.

stops scaling, and on CORE2 they move the parallel-efficiency knee out by a factor of 1.26–1.29.

|     | configuration                 | vertices/rank | SSH % | best solver       | whole-step |
|-----|-------------------------------|---------------|-------|-------------------|------------|
| CPU | CORE2, 512 ranks (production) | 248           | 9.9   | <code>pcsi</code> | −6.30 %    |
|     | CORE2, 864 ranks              | 146           | 18.9  | <code>pcsi</code> | −13.10 %   |
|     | fArc, 2048 ranks              | 312           | 22.8  | <code>oati</code> | −13.28 %   |
|     | DARS, 8192 ranks              | 385           | 6.4   | <code>pcsi</code> | −4.04 %    |
| GPU | CORE2, 2 nodes                | 15857         | 25.8  | <code>oati</code> | −5.32 %    |
|     | CORE2, 16 nodes               | 1982          | 30.5  | <code>oati</code> | −6.73 %    |
|     | fArc, 8 nodes                 | 19949         | 37.7  | <code>oati</code> | −7.32 %    |
|     | DARS, 16 nodes                | 49380         | 7.4   | <code>oati</code> | −1.00 %    |

### 3.2. Which solver

`oati` wins at every configuration measured in range on both backends and tracks the baseline’s iteration count everywhere, which makes it the safer default. `pcsi` converges faster where its Chebyshev interval is accurate — it is the best solver on CORE2 CPU — but that interval rests on an eigenvalue estimate that was poor on fArc, where `pcsi` costs 8–10% on both backends while needing 376 iterations against 212. `pipecg` is indistinguishable from `cg2` on this machine, because no `MPI_Iallreduce` progression is available on any Levante stack we tested.

P-CSI performs no reduction per iteration at all, its only global communication being the convergence test every  $K$  iterations. Raising  $K$  from 1 to the shipped default of 5 more than doubles the whole-step gain, from −4.86% to −11.11% at CORE2 864 ranks. Beyond 5 the return flattens and reappears as wasted work:  $K = 20$  buys 1.2 percentage points more while overshooting the tolerance by 6.7% more iterations.

## 4. Where the step actually goes

Figure 2 resolves the step into compute and MPI wait. Wait accounts for 51% of the step on CORE2 at 864 ranks and 53% on fArc at 2048: the model is synchronisation-bound at these rank counts. The SSH solve is the single largest source of wait, but roughly 90% of the solve’s own cost *is* wait — its arithmetic is 1.0 ms on CORE2 and 2.5 ms on fArc.

Compute is unchanged between baseline and variant (23.5 to 23.1 ms on CORE2), so the variants buy nothing arithmetically and the entire gain is synchronisation. Only 5.8 of the 9.5 ms saved lies

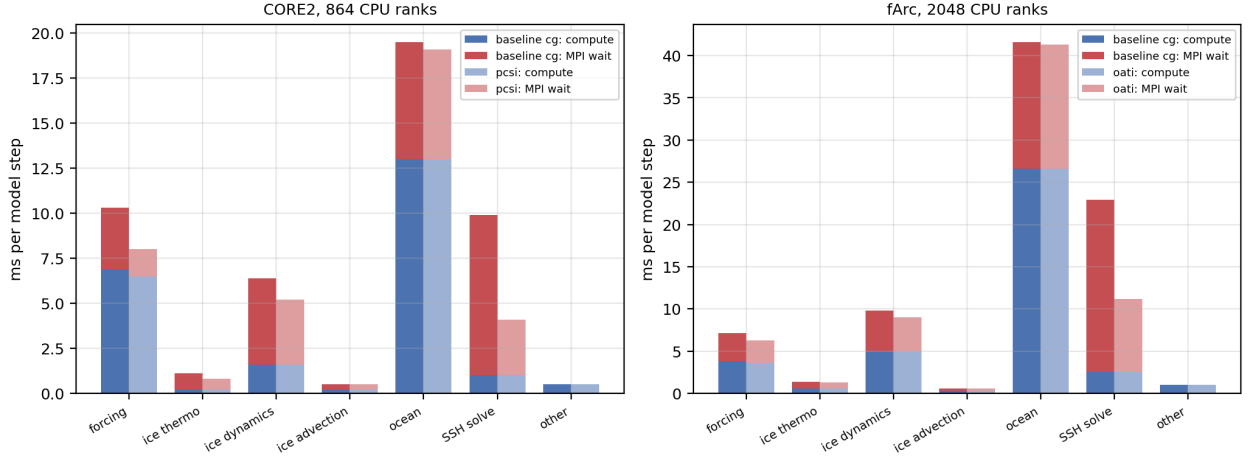


Figure 2: Per-phase compute and MPI wait, baseline against the best solver, from per-rank wall clocks with wait captured inside MPI by PMPI interposition. About half of each step is wait. The SSH bar is almost entirely wait, and the variant also shortens the wait in forcing and ice dynamics, phases whose own MPI call counts are unchanged.

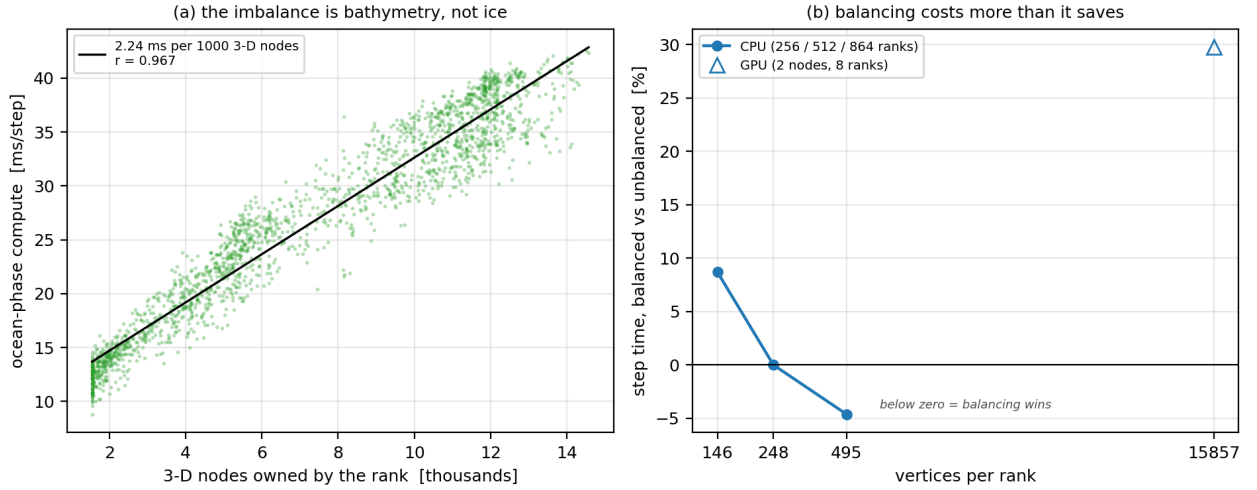


Figure 3: (a) Ocean-phase compute against the number of 3-D nodes each rank owns, all 2048 fArc ranks. (b) Step time with a vertically balanced partition relative to a surface-balanced one, at the same rank count in the same allocation.

inside the solver: 3.7 ms, or 39 %, appears in phases whose MPI call counts and compute times are identical. Removing synchronisation points lets neighbouring phases stop absorbing skew, which is why the whole-step gain exceeds the solve’s own share of the step rather than merely matching it.

## 5. Load imbalance

The ocean phase is the largest block of real computation, and its per-rank cost varies by a factor of 4.8. A bulk-synchronous phase costs its slowest rank, so this is a direct tax: 5.2 ms of a 48.2 ms step on CORE2 and 15.9 ms of 84.7 ms on fArc.

Figure 3(a) identifies the cause. Ocean compute correlates with the number of 3-D nodes a rank owns at  $r = 0.967$ , and with the number of 2-D nodes at  $r = 0.003$ . The partition balances surface nodes to within 1 %, but the work is per water column, and 3-D nodes span a factor of 9.4. The imbalance is bathymetric. Solver wait correlates with 3-D nodes at  $-0.771$ : deep-column ranks wait least because they are the stragglers, so the ranks everyone waits for are the deep ones, not the ice-covered ones.

Balancing the vertical distribution does not pay. Weighting the partitioner by 3-D as well as 2-D nodes removes the imbalance completely (spread  $9.6\times \rightarrow 1.05\times$ ) but forces the partitioner to cut

about ninety times more edges. The resulting fragmentation costs more compute than the balance recovers: on CPU it is worth  $-4.6\%$  at 495 vertices per core, breaks even near 250 and costs  $+8.7\%$  at 146; on GPU it costs  $+29.7\%$ , since the device is far more sensitive to memory-access locality. Note that halo node counts understate the damage — they grow  $40\%$  where the edge cut grows ninetyfold — so the cut, not the halo, is the quantity to watch.

## 6. Status and limitations

CPU and GPU coverage is complete for CORE2, fArc and DARS. NG5 has no valid CPU row: at  $\geq 4096$  ranks the model reaches its own vertical-CFL blow-up guard between steps 150 and 175. That limit is documented for this mesh, timestep and cold start, and Fortran reaches it at step 203 against the port's 200, so measuring NG5 requires either a window shorter than about 120 steps or a smaller timestep. NG5 on GPU rests on a single 16-node configuration.

All numbers come from one machine and a 300-step window. The default solver is unchanged: the switch selects a variant only when set, and which variant to adopt — together with the value the widened guard threshold should ship with — remains open.